Evidence before confidence.

Ayaan Khan | MiraFlores case study | Prepared for GreenGrowth CPAs

Live prototype: ayaan-27.github.io/Miraflores_Project/
Source and README: github.com/ayaan-27/Miraflores_Project

1. Three problems, one choice

A. Month-end evidence scattered across transactions and documents. The hypothesis is that small-business bookkeeping reviewers spend time finding receipts, checking ambiguous matches, and asking clients the same questions repeatedly. Reviewers and clients bear the cost: repeated requests, delayed close, and weak traceability. I have not observed GreenGrowth's internal workflow or interviewed its staff; this is an industry workflow hypothesis. My motivation comes from the provenance problem in my own work: my resume describes building a Ricoh service-document assistant that preserves step-to-image references. A useful answer needs inspectable evidence.

B. Business users cannot confidently answer questions across many databases. My resume describes a Ricoh text-to-SQL system spanning 50+ relational databases and a curated knowledge graph. That work motivates the problem of inconsistent metric definitions and fragmented access for operational users. Left unresolved, users wait for specialists or act on incomparable numbers. My prior work supports the existence of this class of problem; it does not establish a new market's demand or prove every generated answer is reliable.

C. Teams cannot safely share enough representative data to evaluate models. My resume describes a conditional-GAN synthetic-data product and a sensitive-information redaction system at Mphasis. Those projects motivate a data-sharing problem experienced by model builders and data owners: restricting access slows evaluation, while careless sharing can expose sensitive information. I would still need to establish which data constraints matter for a specific new customer.

Choice: A, a narrow evidence-review workflow. It combines my interest in traceable systems with a concrete accounting use case. A reviewer can inspect a transaction, see why it is unresolved, and leave with a specific action. That makes a useful first slice testable without access to production systems, specialist tax rules, or a paid API. B needs validated schemas and domain semantics; C needs a credible privacy-and-utility evaluation. Both are worthwhile, but would be harder to demonstrate honestly in this exercise. This prioritization is about the smallest defensible experiment, not a claim that A has the largest market.

Evidence boundary. The resume is contextual evidence supplied by me, not independently verified research. All prototype records are synthetic. Claims about accounting pain, frequency, willingness to pay, and GreenGrowth's workflow remain assumptions. No customer impact or time savings are claimed.

2. Approach and smallest useful step

Today, as a hypothesis: a reviewer exports transactions, searches a receipt folder or spreadsheet, records exceptions, and emails a client. Existing accounting tools may already address much of this. A better outcome would be a shorter, more precise unresolved-item queue with traceable decisions and fewer duplicate requests, while keeping professional judgment with the reviewer.

First step: import one bank CSV and one receipt-metadata CSV; find candidate evidence; explain exceptions; record a human decision; export a review register and client request. LedgerLens is an evidence triage tool, not a bank balance reconciliation, expense-policy checker, tax adviser, or completed audit. No ledger entry is posted.

Credible alternative: use a standardized spreadsheet template and a client-request checklist. This is cheaper to adopt and may be sufficient at low volume. I chose a small browser application to test whether consistent matching rules, one-receipt-per-charge enforcement, and per-item provenance reduce review effort. If a template performs equally well, a new product is not justified. Another alternative, adding a feature inside an incumbent accounting platform, could offer better distribution; platform access and unmet needs must be researched before building integrations.

Why no runtime AI: the core requirement is exact signed amounts, constrained date comparisons, and explainable exceptions. Conventional rules are easier to audit here than an LLM-generated confidence score. Amounts are parsed into integer cents; a partial vendor match is never automatically confirmed. AI may later help extract receipt fields or suggest merchant aliases, but extraction would need cited source regions, measured error rates, privacy controls, and human verification before it could affect matching.

Broader opportunity: a persistent, permissioned review workspace across clients, eventually connected to existing bookkeeping exports. The current slice tests whether an exception and its next action are useful enough to earn that expansion. It intentionally avoids OCR, integrations, and autonomous posting before validating the workflow.

3. What the prototype actually does

Input > decision > output. Twelve synthetic bank transactions and ten synthetic receipt-index records load automatically. Users can instead import their own structured CSV pair. A candidate must match the exact signed USD amount, fall within a configurable date tolerance (default ±3 days), and have at least 50% normalized merchant-token overlap. Only a unique exact merchant candidate with no competing transaction is automatically marked matched. Shared receipts, multiple candidates, duplicate bank fingerprints, and partial vendor names require review; no candidate means missing evidence.

The reviewer may confirm a candidate, exclude an item with an explanation, or mark evidence requested. Every action requires a note. A receipt cannot be allocated twice, and requested evidence remains unresolved. The underlying matching reason survives a reviewer decision. A downloadable CSV keeps every bank row, rule, receipt reference, note, and decision timestamp; a JSON snapshot also preserves imported metadata. An editable request draft lists only unresolved items. Downloads are real; emails are not sent.

Representative case: Blue Bottle Coffee, B004, $48.50 has one same-day receipt indexed as Blue Bottle. The name overlap produces a review item, not an automatic match. A reviewer inspects the metadata reference, verifies the underlying document outside this prototype, adds a note, and confirms R004. The initial summary changes from 6 of 12 accounted for and $593.50 unresolved to 7 of 12 and $545.00. Those figures describe only synthetic data; they do not measure productivity gains.

Difficult cases: Staples B005 and B006 compete for R005. Both require review. Confirming R005 for one makes a second assignment fail with an explicit error. Harbor Print has two eligible receipts and remains ambiguous. A negative $45 credit memo demonstrates signed refund matching. Importing the supplied invalid bank file fails on duplicate ID B001 and preserves the existing review. Separate tests exercise impossible dates and invalid amounts.

Implemented: real CSV parsing and validation; deterministic matching; explained queue and search; adjustable date tolerance; reviewer notes and undo; one-to-one allocation; editable request and CSV/JSON downloads; responsive browser UI. **Synthetic:** client, transactions, receipt metadata, and file names. **Absent/deferred:** actual document upload or OCR, runtime AI, authenticated reviewer identities, persistent or tamper-evident audit storage, snapshot re-import, accounting/email integrations, FX, split/combined payments, and production security controls. The referenced PDFs and images are labels, not real documents.

Architecture: a dependency-free static site. Pure JavaScript functions own validation, matching, decisions, and exports; the browser UI owns state and interactions. No account, backend, model key, or remote font is needed. Imported data remains in page memory; reloading clears it. The host can log ordinary static-file requests. This choice simplifies evaluation and data handling, but is not a production compliance claim.

4. Validation, learning, and disclosure

Checks completed: 13 automated engine tests passed, including exact outputs, cents arithmetic, date boundaries, CSV quoting, ambiguous/duplicate records, signed refunds, invalid inputs, decision constraints, and export formula guards. A Chromium acceptance test exercised real imports and downloads, reviewer decisions, a rejected second receipt allocation, invalid-import state preservation, HTML injection protection, reset, mobile overflow, and keyboard dialog dismissal. No page errors or unexpected external requests occurred during that test. The static production build also passed. These checks demonstrate prototype behavior, not real-world matching accuracy, production security, or an accountant's approval of a document.

What I would learn next: first interview three bookkeeping reviewers and inspect their existing process using consented, redacted examples. Establish whether the actual bottleneck is evidence matching, business-purpose clarification, or client response time. With five to ten willing client datasets, compare a standardized spreadsheet against LedgerLens. Measure median reviewer minutes per 100 transactions, request completeness, duplicate requests, and false automatic matches adjudicated by a reviewer. Report denominators and uncertainty; synthetic tests cannot estimate these outcomes.

Advance or stop: a provisional pilot goal is at least 25% less reviewer time without increasing incorrect evidence assignments. This is a decision threshold to test, not an achieved result. Stop or narrow the product if reviewers already solve this well in existing tools, matching is not a meaningful time sink, structured receipt indexes take more effort than they save, or the spreadsheet performs just as well. Any incorrect automatic assignment should trigger rule analysis; a human-reviewed result is never promoted to automatic certainty without evidence.

Next engineering step, conditional on demand: replace manual receipt-index preparation with reviewed document extraction, then add authenticated sessions and durable decision history. Support split payments only after learning their frequency. Integrations and privacy/access controls precede any production-client trial. Reproducible sample inputs, expected outputs, setup instructions, and implementation boundaries are in the README.

AI and reuse disclosure: an AI coding assistant helped interpret the brief, propose the scope, generate the interface, matching code, tests, and this written draft, and run the checks. The resume informed problem choices; no employer code, client records, or prior application implementation was reused. Runtime matching is conventional software, not an AI model. Test coverage and source inspection were used to check generated work, including arithmetic, decision conflicts, and adversarial input. The matching functions and regression tests are separated from the UI to support explanation and adaptation during the live session.

Time and delivery: approximately 25 minutes of AI-assisted implementation, writing, testing, and packaging in this session; candidate review and video recording are not included. The prototype is published on GitHub Pages, and the deployment workflow verified the public assets against the tested build. A walkthrough outline is included; the candidate will record the video separately. The next investigation is the reviewer interviews and spreadsheet comparison described above.